

Vježba 10

Aktivacija Pull up otpornika na priključku mikroupravljača

```
.def temp =r16
```

Init:

```
ser temp  
out DDRB,temp  
ldi temp,0b01111111  
out DDRD,temp  
clr temp  
out PortB,temp  
ldi temp,0b10000000  
out PortD,temp
```

Main:

```
in temp,PinD  
lsr temp (x8)  
out PortB,temp  
rjmp Main
```

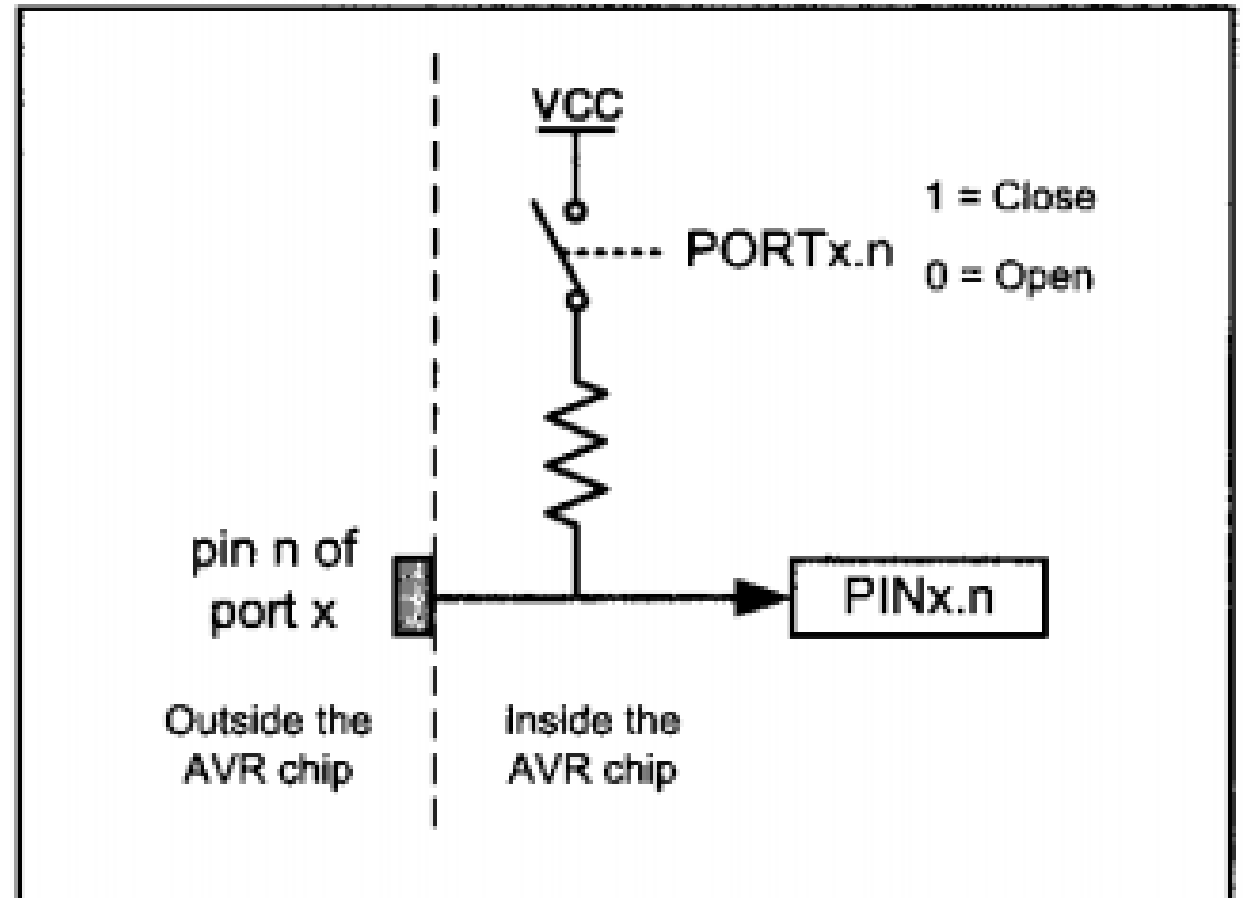


Figure 4-4. The Pull-up Resistor

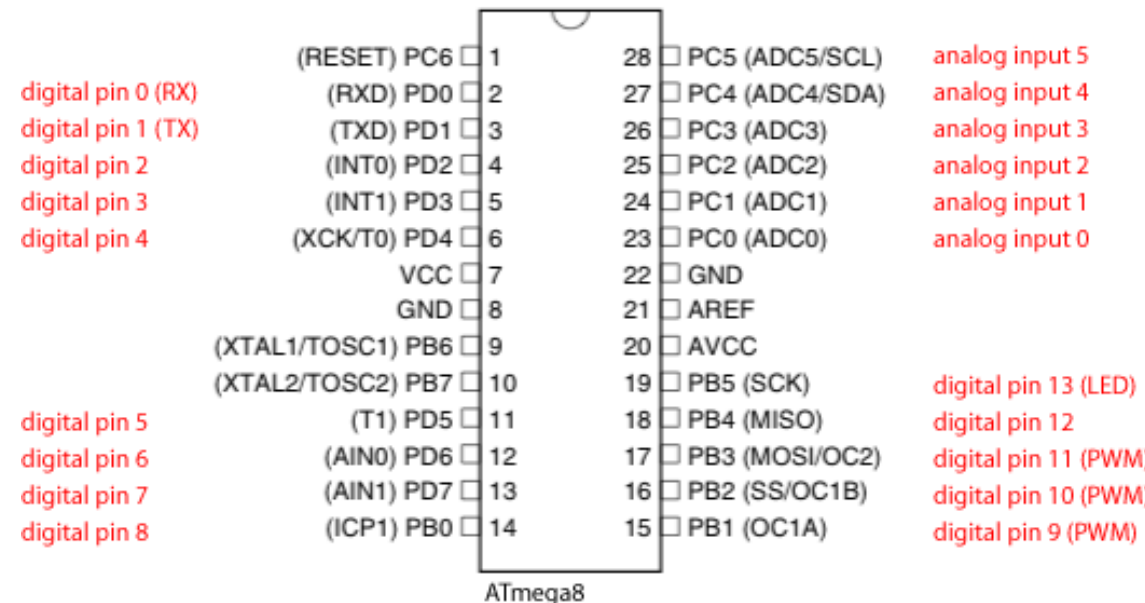
Zadatak 1.

- Programiraj mikroupravljač ATMEGA 328p na način da čita stanja s dva ulaza
 - digitalnog priključka 11 (realiziran s PULL up otpornikom)
 - digitalnog priključka 12 (realiziran bez PULL up otpornikom)

te ispisuje stanja na izlaze (stanje izlaza mijenja se sukladno stanju na ulazu) (izlazni priključci proizvoljno odabrani)

Arduino Pin Mapping

www.arduino.cc



Zadatak 1. - Rješenje

```
.def temp =r16
Init:
ldi temp,0b11100111
out DDRB,temp
ldi temp,0b11111111
out DDRD,temp
    ldi temp,0b00001000
out PortB,temp
    ldi temp,0b00000000
    out PortD,temp
Main:
    in temp,PinB
    out PortD,temp
rjmp Main
```

Directives

- assembly sintaksa se sastoji od:
 - labels,
 - **directives**,
 - instructions,
 - and comments
- Directive su instrukcije koje assembler koristi za automatizaciju procesa montaže i za poboljšanje čitljivosti programa. Primjeri uobičajenih asemblerskih direktiva su ORG (ishodište), EQU (izjednačavanje) i DS. B (definirajte prostor za bajt). ...
- Strojne instrukcije generiraju strojni kod, a asemblerske direktive ne (one samo kontroliraju tok koda).

Assembly directives

Directive	Parameter(s)	Description
Listing		
.LIST		Switches the output in the list file on
.NOLIST		Switches the output in the list file off
Source code origin		
.INCLUDE	(filename)	Assembles the code in the file
Target assembling segments		
.CSEG		Assembles into the program flash segment, address counter: Program Counter PC
.ESEG		Assembles into das EEPROM segment, only labels and .ORG-/.DB-/.DW directives allowed, separate EEPROM address counter
.DSEG		Assembles into the SRAM segment, only labels and .ORG-/.BYTE-/.WORD directives allowed, separate address counter
Address manipulation		
.ORG	Address	Sets the address counter in the active segment, forward to the desired address
.BYTE	N	Reserves N bytes and increases the address counter by N
.WORD	N	Reserves N words and increases the address counter by 2*N bytes

Assembly directives

Tables		
.DB	b1,b2,..bn "(Text)" 'c','h','a','r'	Inserts the bytes b1 to bn resp. the Ascii values of the text and characters into the code or EEPROM segment, code segment: requires an even number of bytes, otherwise a Null byte is inserted!
.DW	w1,w2,..wn	Inserts the double byte value w1 to wn into the code or EEPROM segment
Symbols, names		
.DEF	Name = Rn	Assigns the name to the register
.EQU	Name = value	Assigns the constant value to the named constant, no later re-assignment possible
.SET	Name = value	Assigns the value to the named variable, later re-assignments possible
.UNDEF	Name	Removes the assignment of a register, constant or variable
Macros		
.MACRO	Name, Parameter	Starts a macro of that name and with the given parameters
.ENDMACRO .ENDM		Ends the macro definition

Assembly directives

Type definitions		
.DEVICE	"Type name"	Switches instruction checks on, use of undefined instructions in that device type causes an error
Messages, error handling		
.MESSAGE	"TEXT"	Outputs the given text
.ERROR	"TEXT"	Causes an error and outputs the given text
Conditional assembly		
.EXIT		Ends assembling, further source code ignored
.IF	Condition	Assembles the following code only if the condition is true, requires .ENDIF
.IFDEF	Symbol	Assembles the following code if the symbol is defined, requires .ENDIF
.IFNDEF	Symbol	Assembles the following code if the symbol is not defined, requires .ENDIF
.ELSE		Assembles the following code if the condition in .IF is false or the symbol in .IFDEF is not defined or the symbol in .IFNDEF is defined
.ELIF	Condition	Assembles the following code if the condition in .IF is not true and the condition in .ELIF is true
.ENDIF		Ends .IF, .IFDEF, .ELSE und .ELIF directives

Assembly directives

gavrasm specialties		
.DB	%YEAR% %MONTH% %DAY% %HOUR% %MINUTE% %SOURCE%	Inserts current date and filename in ASCII format
.IFDEVICE	"Device name"	Conditioned assembly if the device name is valid, requires previous .DEVICE or .INCLUDE "e;*def.inc"e

.MACRO (1/4)

- Definicija Macro

```
.macro    myMacro
    ...
; macro code
.endmacro
```

PRIMJER 1. Inicijalizacija STACK memorije

```
.macro    setStack
    ldi    r16,LOW(RAMEND)
    out    SPL,r16
    ldi    r16,HIGH(RAMEND)
    out    SPH,r16
.endmacro
```

.MACRO (2/4)

- Pozivanje MACRO (bez parametara)

```
start: setStack          ; initialize stack pointer
      ...                ; program code
loop:  rjmp  loop         ; infinite loop
```

.MACRO (3/4)

- Pozivanje MACRO s PARAMETRIMA

PRIMJER 1.

- Definicija Macro

```
.macro    setStack
        ldi        r16,LOW(@0)
        out        SPL,r16
        ldi        r16,HIGH(@0)
        out        SPH,r16
.endmacro
```

- Pozivanje

```
start:    setStack RAMEND        ; initialize stack pointer
        ...                      ; program code
loop:     rjmp     loop          ; infinite loop
```

.MACRO (4/4)

- Pozivanje MACRO s PARAMETRIMA

PRIMJER 2.

- Definicija Macro

```
.macro    setStack
        ldi        @1,LOW(@0)
        out        SPL,@1
        ldi        @1,HIGH(@0)
        out        SPH,@1
.endmacro
```

- Pozivanje

```
start:    setStack RAMEND,r16        ; initialize stack pointer
        ...                          ; program code
loop:     rjmp     loop              ; infinite loop
```

Zadatak 2.

- Izradi macro direktivu koja izvršava sljedeću matematičku funkciju

$$Y = (a + b + c) * (c - a)$$

Rezultat spremi u registar R5

Zadatak 2 - rješenje

```
.MACRO  jednadzba
```

```
mov  R19,@0
```

```
add  R19,@1
```

```
adc  R19,@2
```

```
mov  R20,@2
```

```
sub  R20,@0
```

```
mul  R19,R20
```

```
mov  R5,R0
```

```
.endmacro
```

```
loop:
```

```
ldi  R16,0x01
```

```
ldi  R17,0x02
```

```
ldi  R18,0x03
```

```
ldi  R19,0x04
```

```
jednadzba  r16, r17 ,r18;
```

```
rjmp  loop
```

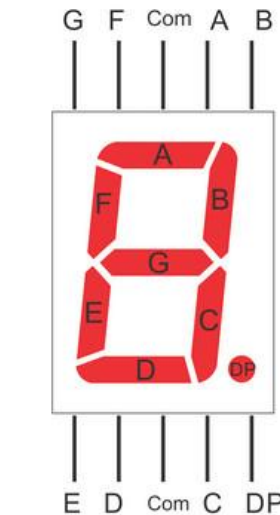
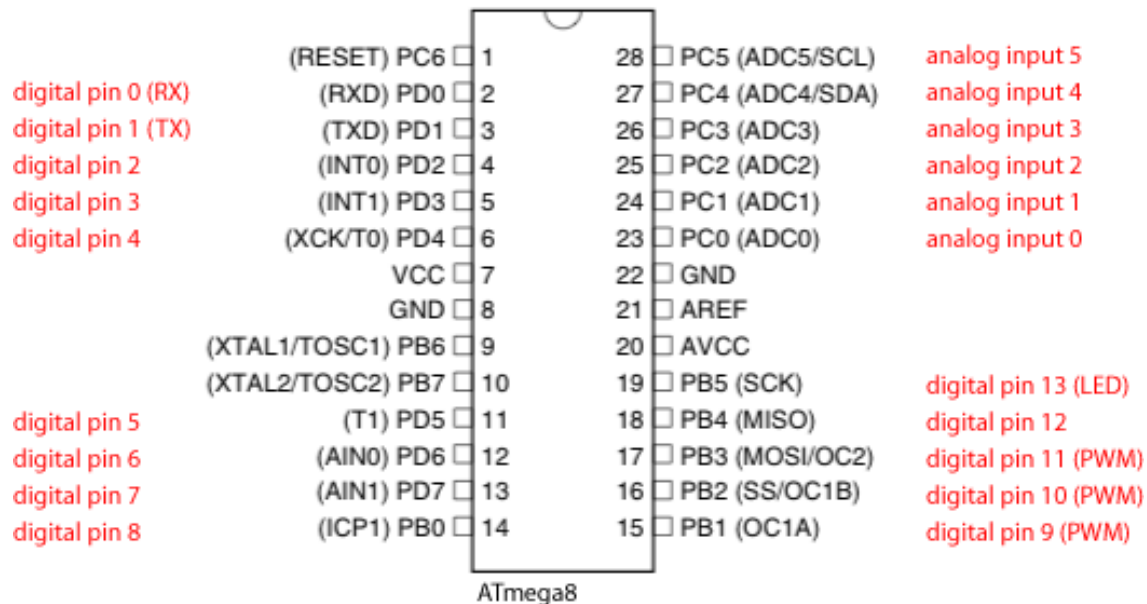
Conditional assembly directive

```
.macro      setStack
    .if @0>RAMEND
        .error "Value greater than RAMEND used for setting stack"
    .else
        ldi    @1,LOW(@0)
        out    SPL,@1
        ldi    @1,HIGH(@0)
        out    SPH,@1
    .endif
.endmacro
```

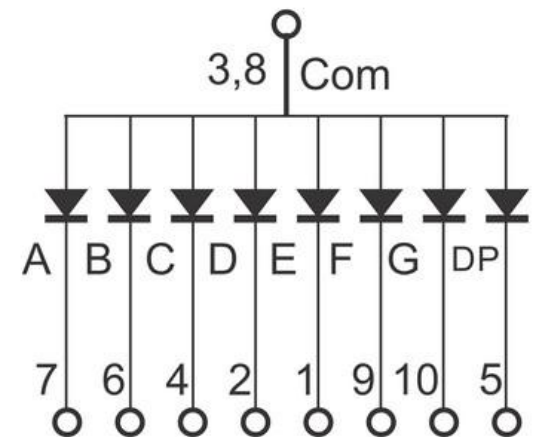

Primjer 1. 7 segmentni pokazivač

- Programiraj mikroupravljač da broji od 0 do 9 te broj prikazuje na 7 seg. pokazivaču. Za pohranu kombinacija brojeva za prikaz koristi tablicu.

Arduino Pin Mapping



7 Segment Display - Pin Out Diagram



www.circuitstoday.com

Primjer 1. 7 segmentni pokazivač - rješenje

```
.def    temp    =r22
init:
    ser    temp
    out    DDRD,temp
clr     temp
    out    PORTD,temp
call    delay
resetadrese:
ldi     ZL,LOW(2*tablica); load 2*var
ldi     ZH,HIGH(2*tablica); into Z pointer
loop:
lpm     R4,Z+
out     PORTD,R4
call    delay

mov     r23,r4
CPI     R23,0x6f
brne    loop
jmp     resetadrese

tablica:
.db      0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6f

delay:
LDI     R16,    127;1 strojni ciklus
Delay1:
LDI     R17,    125;1 strojni ciklus
Delay2:
LDI     R18,    250    ; 1 strojni ciklus

Delay3:
DEC     R18          ; 1 strojni ciklus
NOP          ; 1 strojni ciklus
BRNE    Delay3      ; 2 strojni ciklus

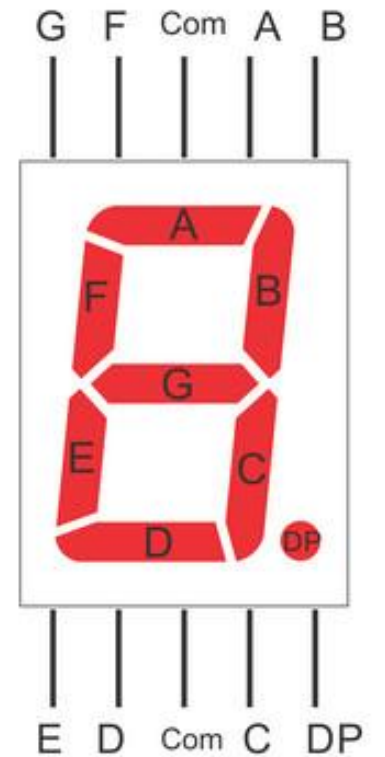
DEC     R17          ; 1 strojni ciklus
BRNE    Delay2      ; 2 strojni ciklus

DEC     R16          ; 1 strojni ciklus
BRNE    Delay1      ; 2 strojni ciklus
ret
```

Zadatak 1.

- Realizira brojanje pomoću 7 segmentnog pokazivača na način da broji unatrag od 9 do 0.
- Pinovi pokazivača su spojeni na sljedeći način

7segin	h	g	f	e	d	c	b	a
Arduino pin	11	10	9	8	7	6	5	4
Adresa pina na portu	PB3	PB2	PB1	PB0	PD7	PD6	PD5	PD4



Zadatak 1. - rješenje

```
.def    temp    =r22

init:

    ser    temp

    out    DDRD,temp

    clr    temp

    out    PORTD,temp

    ser    temp

    out    DDRB,temp

    clr    temp

    out    PORTB,temp

;call delay

resetadrese:

    ldi    ZL,LOW(2*tablica+10); load 2*var
    ldi    ZH,HIGH(2*tablica); into Z pointer

loop:

    dec    R30

    lpm    R4,Z

    mov    R19,R4

    mov    R20,R4

    swap   R19

    andi   R19,0b111110000

    swap   R20

    andi   R20,0b00001111

    out    PORTD,R19

    out    PORTB,R20

    call   delay

    mov    r23,r4

    CPI    R23,0x3F

    brne   loop

    jmp    resetadrese

tablica:

    .db    0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6f

delay:

    LDI     R16,    127;1 strojni ciklus

Delay1:
```

```
LDI     R17,    125;1 strojni ciklus

Delay2:

    LDI     R18,    250    ; 1 strojni ciklus

Delay3:

    DEC     R18            ; 1 strojni ciklus
    NOP                    ; 1 strojni ciklus
    BRNE    Delay3        ; 2 strojni ciklus

    DEC     R17            ; 1 strojni ciklus
    BRNE    Delay2        ; 2 strojni ciklus

    DEC     R16            ; 1 strojni ciklus
    BRNE    Delay1        ; 2 strojni ciklus

    ret
```

Zadatak 2.

- Realiziraj brojač koji broji koliko puta je pritisnuto tipkalo te taj rezultat prikazuje na 7 segmentnom pokazivaču. Broji od 0-9 te se zatim resetira.
- Tipkalo je spojeno na digitalni priključak 8

Zadatak 2. - rješenje

```
.def    temp    =r22
init:
clr     temp
out     DDRB,temp
sbi     PORTB,0
        ser     temp
        out     DDRD,temp
        clr     temp
        out     PORTD,temp
;call delay
resetadre:
ldi     ZL,LOW(2*tablica+1); load 2*var
ldi     ZH,HIGH(2*tablica+1); into Z pointer
ldi     R16,0x3f
mov     r4,R16
out     PORTD,R4
loop:
        SBIS    PINB,0
        call    ispisbroja
        mov     r23,r4
        CPI     R23,0xFF
        brne    loop
        jmp     resetadre

ispisbroja:
lpm     R4,Z+
out     PORTD,R4
call    delay
ret

tablica:
.db     0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6f,0xFF

delay:
        LDI     R16, 127;1 strojni ciklus
Delay1:
        LDI     R17, 125;1 strojni ciklus
Delay2:
        LDI     R18, 250      ; 1 strojni ciklus
Delay3:
        DEC     R18           ; 1 strojni ciklus
        NOP                     ; 1 strojni ciklus
        BRNE    Delay3       ; 2 strojni ciklus

        DEC     R17           ; 1 strojni ciklus
        BRNE    Delay2       ; 2 strojni ciklus

        DEC     R16           ; 1 strojni ciklus
        BRNE    Delay1       ; 2 strojni ciklus
        ret
```